

Roognis AI

Complete System Design Document

HLD + LLD v3 · All critical fixes applied · Investor-demo ready

Roognis AI — Verified Final HLD

All 8 review fixes applied · Demo-safe · Cloud-portable · Ready for LLD

- ✓ 8/8 FIXES APPLIED

✓ COMFYUI ADDED

✓ CHROMADB PERSISTENT

✓ RBAC + HTTPONLY JWT

✓ PARENT→STUDENT RELATION

✓ ANALYTICS EVENT FLOW

✓ RAG CHUNKING DEFINED

 DOCKER-COMPOSE UP → FULL SYSTEM

CLIENTS · BROWSER

NEXT.JS · :3000

Web App — Role-Based Portals

Student: AI chat (SSE stream) · image gen · video playback · feedback

Teacher: batch analytics · query trends · intervention alerts

Parent: child progress · subject scores · attendance

JWT in HttpOnly cookie · 24h expiry

EventSource API for streaming

↓ HTTPS · Cookie: jwt=token>

API GATEWAY

TRAEFIK · :80

API Gateway

JWT middleware (shared secret with Auth Service)

/api/auth/* → auth:3001 · /api/ai/* → ai:3002

/api/rag/* → rag:3003 · /api/analytics/* → analytics:3004

CORS: localhost:3000

Health checks on all services

JWT payload: { userId, role: "student"|"teacher"|"parent", schoolId, studentId? }

Every service checks **role** before responding. Parent gets **studentId** of their child injected at gateway level via parent_student lookup.

↓ Internal Docker network

MICROSERVICES · DOCKER COMPOSE

:3001 · AUTH_DB SCHEMA

Auth Service

Register · Login · JWT issue

Users · Roles · Schools

parent_student relation table

:3002 · AI_DB SCHEMA

AI Service

Text: → RAG retrieve → Ollama stream

Image: → ComfyUI API

Video: serve pre-generated file

Conversation history in ai_db

Loading SSE event on cold start

:3003 · RAG_DB SCHEMA

RAG Service

Upload PDF → chunk → embed → ChromaDB

/retrieve → top-5 chunks for AI Service

nomic-embed-text via Ollama

512 tok chunks · 50 tok overlap

10 NCERT chapters seeded on boot

:3004 · ANALYTICS_DB SCHEMA

Analytics Service

Scores · Attendance · Query counts

Feedback loop · Intervention flags

Fire-and-forget event POST

Teacher marks attendance via API

Demo data seeded on boot

AI chat request flow: Student → Traefik → AI Service → **RAG /retrieve?q=...** (top-5 chunks) → AI Service builds prompt (chunks + history) → **Ollama /api/generate (stream)** → SSE tokens → browser renders live | AI Service fires **POST analytics/event (async, no await)**

Image request flow: Student → AI Service → **ComfyUI /api/prompt** (loads SD on demand, unloads after) → image saved to Docker volume → URL returned to browser

↓ Read / Write

DATA LAYER

POSTGRESQL · :5432

Primary Database

4 isolated schemas

auth_db · ai_db · rag_db · analytics_db

Each service owns + migrates its schema

CHROMADB · :8000

Vector Store

Collections per subject

Semantic search for RAG

Persistent Docker volume (no data loss)

DOCKER VOLUME · /DATA

File Storage

/data/uploads — PDFs

/data/images — generated

/data/videos — pre-generated clips

→ S3 post-funding via ENV swap

↓ HTTP API

LOCAL AI LAYER · ALL ON LAPTOP

OLLAMA · :11434 · ALWAYS LOADED

Qwen2.5 / DeepSeek

COMFYUI · :8188 · ON DEMAND

Stable Diffusion

DOCKER VOLUME · PRE-GENERATED

Video Clips

Text inference · RAG chat
nomic-embed-text for RAG embeddings
Pulled on first startup via init script

Educational image generation
Loads only when image requested
Unloads after — frees VRAM

5 pre-made educational clips
Served from disk instantly
Live gen enabled post-funding

Cloud migration (post-funding): All services read config from ENV vars. No hardcoded localhost anywhere. Ollama URL → Anthropic/OpenAI API · Docker volume → S3 · PostgreSQL → RDS · ChromaDB → Pinecone · Docker Compose → Kubernetes. Each swap is one ENV var change.

● Frontend

● Traefik

● Services

● AI Service

● RAG Service

● Databases

● File Storage

● Ollama (LLM)

● ComfyUI (Image)

LLD v3 — Auth Service 3001 · auth_db

v3: /health endpoint added · register restricted to student/parent only · JWT middleware moved here from Traefik

API ENDPOINTS

METHOD + PATH	REQUEST BODY	RESPONSE	AUTH?
GET /health W1 NEW	—	{ status:"ok" } · 200 · used by docker-compose healthcheck	No
POST /api/auth/register W2 FIXED	{ name, email, password, role, schoolId }	201 · { userId, role } · only role:"student" or role:"parent" accepted. 400 if role:"teacher"	No
POST /api/auth/login	{ email, password }	200 · Sets HttpOnly cookie jwt (24h) · rate limited 10/min	No
POST /api/auth/logout	—	200 · Clears cookie	Yes
GET /api/auth/me	—	{ userId, name, email, role, schoolId }	Yes
POST /api/auth/link-parent	{ parentId, studentId }	200 · parent_student row created	Yes · teacher
GET /api/auth/parent/:id/students	—	[{ studentId, name }]	Yes · parent/teacher

⚠ W2: Teacher accounts ONLY created via seed script — not registerable. Attempting role:"teacher" via register returns 400 Bad Request.

JWT MIDDLEWARE — APPLIED IN EVERY SERVICE C1 NEW

```
// middleware/auth.js — shared pattern, copy to each service
const jwt = require('jsonwebtoken');

module.exports = function requireAuth(req, res, next) {
  const token = req.cookies?.jwt;
  if (!token) return res.status(401).json({ error: 'Unauthorized' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (e) {
    res.status(401).json({ error: 'Invalid token' });
  }
};

// Role guard helper
module.exports.requireRole = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ error: 'Forbidden' });
  next();
};

// Usage in any service:
// app.use('/api/ai', requireAuth);
// router.post('/score', requireAuth, requireRole('teacher'), handler)
```

✔ JWT is verified at each service directly. Traefik handles routing + CORS only — no JWT plugin needed.

DB SCHEMA — AUTH_DB

schools		
id	UUID PK	seeded: DEMO_SCHOOL_ID from .env
name	VARCHAR	
users		
id	UUID PK	
name	VARCHAR	
email	VARCHAR	UNIQUE
password_hash	VARCHAR	bcrypt cost 10
role	ENUM	student teacher parent
school_id	UUID FK	→ schools.id
created_at	TIMESTAMP	DEFAULT now()

INTERNAL LOGIC

Login: - Find user by email - bcrypt.compare(password, hash) - If parent → fetch studentIds from parent_student - jwt.sign({ userId, role, schoolId, studentIds? }, JWT_SECRET, { expiresIn:'24h' }) - Set-Cookie: jwt=token; HttpOnly; SameSite=Strict; Path=/; Max-Age=86400
Register (W2 fix): - Validate role is "student" or "parent" only - Return 400 if role is "teacher" or anything else - Teacher accounts exist only in seed script

parent_student

parent_id	UUID FK	→ users.id
student_id	UUID FK	→ users.id
PRIMARY KEY	(parent_id, student_id)	

Rate limit on /login:

express-rate-limit: windowMs=60000, max=10

DEMO SEED SCRIPT

```
// runs on boot if users table is empty
School: Demo School (id = DEMO_SCHOOL_ID from .env)

Teacher: ← created ONLY via seed, not via /register
email: teacher@demo.com pass: demo1234

Students:
arjun@demo.com / demo1234
priya@demo.com / demo1234
rahul@demo.com / demo1234

Parents:
parent1@demo.com / demo1234 → linked to arjun
parent2@demo.com / demo1234 → linked to priya

// Demo login card shown on /login page
```

TECH STACK + ENV

```
Runtime: Node.js + Express | ORM: Prisma | Password: bcryptjs (cost 10) | JWT: jsonwebtoken + cookie-parser | Rate limit: express-rate-limit

DATABASE_URL =postgresql://postgres:${DB_PASSWORD}@postgres:5432/roognis?schema=auth_db
JWT_SECRET =from .env
DEMO_SCHOOL_ID =550e8400-e29b-41d4-a716-446655440000
PORT =3001
```

LLD v3 — AI Service :3002 · ai_db

v3: /health added · video topics endpoint · {HISTORY} format defined · image job timeout cron · RAG retrieve includes schoolId+subject

API ENDPOINTS			
METHOD + PATH	REQUEST	RESPONSE	ROLE
GET /health W1 NEW	—	{ status:"ok" } · 200 · used by docker-compose healthcheck	No auth
POST /api/ai/chat/session	{ subject }	{ sessionId } · creates chat_sessions row	student
POST /api/ai/chat	{ message, sessionId } · max 500 chars	SSE stream → tokens → [DONE] event	student
GET /api/ai/chat/:sessionId/history	—	[{ role, content, timestamp }]	student
POST /api/ai/image	{ prompt }	{ jobId, status:"queued" } · returns immediately	student
GET /api/ai/image/:jobId/status	—	{ status:"queued processing done failed", imageUrl? }	student
GET /api/ai/images/:filename W1 NEW	—	Serves generated image file from /data/images/{filename} . imageUrl in status response is /api/ai/images/{jobId}.png . Returns 404 if job not done yet.	student
GET /api/ai/video/topics W3 NEW	—	[{ topic, filename, subject }] — hardcoded list of 5 available topics. Frontend uses this to populate video picker.	student
GET /api/ai/video/:topic	—	{ videoUrl } · serves pre-generated file from /seed-data/videos/{filename} . seed-data volume mounted read-only in AI service.	student
POST /api/ai/feedback	{ sessionId, messageId, rating:1-5, comment? }	201 · fires async event to Analytics	student

VIDEO TOPICS LIST — HARDCODED W3 DEFINED

```
// GET /api/ai/video/topics - returns this hardcoded list
// Files served from /seed-data/videos/ - volume mounted read-only in AI service
[
  { topic: "photosynthesis",    filename: "photosynthesis.mp4",    subject: "Science" },
  { topic: "fractions",        filename: "fractions.mp4",        subject: "Math" },
  { topic: "water-cycle",      filename: "water-cycle.mp4",      subject: "Science" },
  { topic: "parts-of-speech",   filename: "parts-of-speech.mp4",   subject: "English" },
  { topic: "solar-system",     filename: "solar-system.mp4",     subject: "Science" }
]

// Frontend calls GET /topics first - shows picker - calls GET /video/:topic
```

TEXT CHAT FLOW (SSE)

```
1. Validate: role === student, message ≤ 500 chars, sessionId exists
2. Load last 10 messages from ai_db for this sessionId
3. GET rag:3003/api/rag/retrieve?q={message}&schoolId={req.user.schoolId}&subject={session.subject}&top=5 → top-5 chunks C4 FIXED
4. Build prompt: system prompt with {RAG_CHUNKS} and {HISTORY} substituted
5. Set headers: Content-Type: text/event-stream , Cache-Control: no-cache
6. If Ollama cold start → send event:status\ndata:loading\n\n
7. POST ollama:11434/api/generate (stream:true) → pipe each token as event:token\ndata:{text}\n\n
8. On complete: INSERT both messages to ai_db · fire analytics event (no await)
9. Send event:done\ndata:[DONE]\n\n
```

IMAGE GEN FLOW — ASYNC + TIMEOUT CRON

```
POST /api/ai/image — returns { jobId } immediately. Stores job in image_jobs table with status="queued".

BG Background worker picks up job: enhance prompt → POST comfyui:8188/api/prompt with workflow JSON below → poll GET /history/{promptId} until done → download output image → save to /data/images/{jobId}.png → update job status="done"

GET /api/ai/image/:jobId/status — frontend polls every 3s. Returns { status, imageUrl } when done.

⚡ Timeout Cron: Every 2 minutes — UPDATE image_jobs SET status='failed' WHERE status='processing' AND created_at < now() - interval '5 minutes' . Prevents spinner running forever if ComfyUI crashes.
```

COMFYUI WORKFLOW JSON — POST TO COMFYUI:8188/API/PROMPT

```
// Replace {PROMPT} with the user's prompt text at runtime
{
  "prompt": {
    "4": { "class_type": "CheckpointLoaderSimple",
```

```
"inputs": { "ckpt_name": "v1-5-pruned-emaonly.ckpt" } },

"5": { "class_type": "EmptyLatentImage",
  "inputs": { "width": 512, "height": 512, "batch_size": 1 } },

"6": { "class_type": "CLIPTextEncode",
  "inputs": { "clip": ["4", 1], "text": "{PROMPT}, educational, colorful, diagram style" } },

"7": { "class_type": "CLIPTextEncode",
  "inputs": { "clip": ["4", 1], "text": "ugly, blurry, nsfw, text, watermark, low quality" } },

"3": { "class_type": "KSampler",
  "inputs": { "model": ["4", 0], "positive": ["6", 0], "negative": ["7", 0],
    "latent_image": ["5", 0], "seed": 42, "steps": 20,
    "cfg": 7, "sampler_name": "euler", "scheduler": "normal", "denoise": 1 } },

"8": { "class_type": "VAEDecode",
  "inputs": { "samples": ["3", 0], "vae": ["4", 2] } },

"9": { "class_type": "SaveImage",
  "inputs": { "filename_prefix": "roognis", "images": ["8", 0] } }
}

// After POST → response gives { prompt_id }
// Poll GET /history/{prompt_id} until outputs exist
// Download image from GET /view?filename=roognis_00001.png&type=output
// Save to /data/images/{jobId}.png
```

SYSTEM PROMPT + {HISTORY} FORMAT DEFINED W3 FIXED

```
You are Roognis, an AI tutor for school students.

Rules:
- Answer ONLY based on the provided context below.
- If the answer is not in the context, say:
  "I don't have information on that yet."
- Be concise, friendly, and use simple language
  suitable for school students.
- Never make up facts.
- Format answers with bullet points when listing.

Context:
{RAG_CHUNKS}

Conversation so far:
{HISTORY}

// {HISTORY} format - plain text, last 10 exchanges:
// User: <message>
// Assistant: <message>
// User: <message>
// Assistant: <message>
// ... (up to 10 pairs, oldest first)

// {RAG_CHUNKS} format - concatenated chunk text:
// [1] <chunk text> (source: filename.pdf)
// [2] <chunk text> (source: filename.pdf)
// ...
```

DB SCHEMA — AI_DB

chat_sessions		
id	UUID PK	
student_id	UUID	from JWT
subject	VARCHAR	used for RAG collection targeting
created_at	TIMESTAMP	
messages		
id	UUID PK	
session_id	UUID FK	→ chat_sessions
role	ENUM	user assistant
content	TEXT	max 500 chars for user
created_at	TIMESTAMP	
image_jobs		
id	UUID PK	= jobid
student_id	UUID	
prompt	TEXT	
status	ENUM	queued processing done failed
image_url	VARCHAR	nullable until done
created_at	TIMESTAMP	used by timeout cron
feedback		
id	UUID PK	
message_id	UUID FK	→ messages
student_id	UUID	
rating	INT	1-5
comment	TEXT	nullable
created_at	TIMESTAMP	

TECH STACK + ENV

```
Runtime: Node.js + Express
ORM: Prisma
Background jobs: Simple in-process setImmediate worker (MVP). BullMQ post-funding.
Timeout cron: node-cron · every 2 minutes
SSE: Native Node.js res.write
```

```
OLLAMA_URL =http://ollama:11434
COMFYUI_URL =http://comfyui:8188
RAG_SERVICE_URL =http://rag:3003
ANALYTICS_URL =http://analytics:3004
DATABASE_URL =...?schema=ai_db
FILE_STORAGE_PATH =/data
JWT_SECRET =from .env
PORT =3002
```

LLD v3 — RAG Service & Analytics Service

v3: retrieve endpoint fixed (schoolId+subject) · seed-data folder defined · parent dashboard query param · class assign endpoint · health endpoints added

RAG Service · :3003

API ENDPOINTS		
ENDPOINT		DESCRIPTION
GET /health	W1 NEW	{ status:"ok" } · docker-compose healthcheck
POST /api/rag/upload		Multipart PDF + subject field · returns { documentId, status:"processing" } immediately. 20MB limit — returns 413 if exceeded.
GET /api/rag/upload/:docId/status		Poll embedding progress → { status, chunksProcessed, totalChunks }
GET /api/rag/retrieve	C4 FIXED	?q=query&schoolId=xxx&subject=xxx&top=5 · AI Service passes schoolId+subject. RAG queries school_{schoolId}_{subject} ChromaDB collection. Returns top-5 chunks. NO JWT required — internal call from AI service only, not meant for browser access.
GET /api/rag/documents		List uploaded docs for this school (schoolId from JWT)

UPLOAD FLOW — ASYNC BACKGROUND TASK	
1.	Receive PDF + subject via multipart · reject if >20MB
2.	Save to /data/uploads/{uuid}.pdf
3.	INSERT document row with status="processing"
4.	Return { documentId } immediately — HTTP response done
BG	FastAPI BackgroundTasks picks up:
→	Extract text with PyMuPDF
→	Chunk: 512 tokens, 50 overlap (LangChain RecursiveCharacterTextSplitter)
→	For each chunk: POST ollama/api/embeddings (nomic-embed-text) → 768-dim vector
→	collection.get_or_create("school_{schoolId}_{subject}") → upsert chunk to ChromaDB
→	Update document row: status="done", chunk_count=N

RETRIEVE FLOW C4 FIXED	
1.	Read schoolId and subject from query params (passed by AI Service)
2.	Embed query via ollama nomic-embed-text
3.	ChromaDB similarity search on school_{schoolId}_{subject} collection
4.	Return [{ text, source, score }] top-5

SEED DATA FOLDER C5 DEFINED	
# Repo structure — committed to git	
seed-data/	
pdfs/	
ncert-math-ch1.pdf	# subject: Math
ncert-math-ch2.pdf	# subject: Math
ncert-science-ch1.pdf	# subject: Science
ncert-science-ch2.pdf	# subject: Science
ncert-english-ch1.pdf	# subject: English
videos/	
photosynthesis.mp4	# 30-60s MP4 clips, 720p
fractions.mp4	
water-cycle.mp4	
parts-of-speech.mp4	
solar-system.mp4	

Analytics Service · :3004

API ENDPOINTS		
ENDPOINT		DESCRIPTION
GET /health	W1 NEW	{ status:"ok" } · docker-compose healthcheck
POST /api/analytics/event		Fire-and-forget from other services. schema: { type, studentId, schoolId, subject?, sessionId, metadata:{} } · NO JWT required — designed for internal service-to-service calls. Reachable via Traefik (/api/analytics is routed) but no auth token enforcement acceptable for localhost MVP demo.
POST /api/analytics/attendance		Teacher marks attendance · { studentId, date, present }
POST /api/analytics/score		Teacher enters score · { studentId, subject, score, testDate } · (teacherId, studentId) in class_assignments table
POST /api/analytics/class/assign	NEW	Teacher adds student to their class · { studentId } · teacherId from JWT
GET /api/analytics/student/:id		Full profile: scores, attendance %, count, feedback avg
GET /api/analytics/teacher/dashboard		Only students in teacher's class · schoolId, class_assignments (teacherId from JWT)
GET /api/analytics/teacher/interventions		Students flagged by intervention rule
GET /api/analytics/parent/dashboard	C6 FIXED	?studentId=xxx · required param missing. Ownership check: verify studentId is in req.user.studentIds[] — return 403 if not (prevents parent A from viewing parent B's child). Frontend shows class selector using studentIds[] from JWT
GET /api/analytics/queries/trends		GROUP BY metadata->'subject' on documents table

ANALYTICS EVENT CONTRACT DEFINED	
// All services fire to POST /api/analytics/event	
// Shared schema — all fields required except optional ones	
{	
type:	"chat_message" "image_generated" "feedback_submitted" "intervention_flagged"
studentId:	UUID, // required
schoolId:	UUID, // required
subject:	string, // optional — from chat session subject
sessionId:	UUID, // optional — for chat events
metadata:	{ } // JSONB — extra fields per event type
}	
// Called fire-and-forget: no await, failure is non-fatal	

INTERVENTION RULE	
-- Flag a student for intervention if EITHER:	
-- 1. Low feedback average (poor AI quality perception)	
AVG(rating) < 3.0	
WHERE student_id = ? AND created_at > now() - interval '7 days'	
-- 2. Low engagement (not using the tutor)	


```
# Seed script runs on RAG service boot if documents table empty.
# Reads all PDFs from /seed-data/pdfs/, uploads each to itself via
# POST /api/rag/upload - uses DEMO_SCHOOL_ID from env.
```

✔ PDFs: Download free from ncert.nic.in — small (1-3MB each), public domain.

✔ Videos — use this one-time script to download from [Wikimedia Commons](https://commons.wikimedia.org/wiki/File:Photosynthesis.mp4) (CC licensed):

```
pip install yt-dlp
# Run from repo root - downloads and trims to 60s each
yt-dlp -o "seed-data/videos/photosynthesis.mp4"
"https://commons.wikimedia.org/wiki/File:Photosynthesis.ogv"
yt-dlp -o "seed-data/videos/water-cycle.mp4"
"https://commons.wikimedia.org/wiki/File:The_Water_Cycle.ogv"
# For fractions / parts-of-speech / solar-system:
# Download any CC-licensed clip and rename to match the filename above.
# Alternatively: record a 30s screen-capture of a slide - good enough for demo.
```

DB SCHEMA — RAG_DB

documents			
id	UUID PK		
school_id	UUID		
uploaded_by	UUID	user id	
filename	VARCHAR		
subject	VARCHAR	set by uploader · used for collection name	
status	ENUM	processing done failed	
chunk_count	INT	set when done	
file_path	VARCHAR		
created_at	TIMESTAMP		

PYTHON JWT MIDDLEWARE — FASTAPI W3 NEW

```
# dependencies/auth.py - copy this pattern to the RAG service
from fastapi import Depends, HTTPException, Cookie
import jwt, os
from typing import Optional

# alias="jwt" is required - Cookie(None) uses parameter name as key
# without alias it would look for cookie "jwt_cookie", not "jwt"
def require_auth(jwt_cookie: Optional[str] = Cookie(None, alias="jwt")):
    if not jwt_cookie:
        raise HTTPException(status_code=401, detail="Unauthorized")
    try:
        payload = jwt.decode(
            jwt_cookie,
            os.environ["JWT_SECRET"],
            algorithms=["HS256"]
        )
        return payload # { userid, role, schoolid, studentids? }
    except jwt.InvalidTokenError:
        raise HTTPException(status_code=401, detail="Invalid token")

# Usage in any route:
# @app.post("/api/rag/upload")
# async def upload(user=Depends(require_auth)):
#     school_id = user["schoolid"] ...

# pip install PyJWT
```

Tech: Python · FastAPI · LangChain · PyMuPDF · chromadb SDK · FastAPI BackgroundTasks · PyJWT · SQLAlchemy

DB init: Call `Base.metadata.create_all(engine)` inside FastAPI

`@app.on_event("startup")` — creates the documents table automatically on first boot.

ENV: `OLLAMA_URL` · `CHROMA_URL` · `DATABASE_URL` · `FILE_STORAGE_PATH` · `DEMO_SCHOOL_ID` · `JWT_SECRET` · `PORT=3003`

```
COUNT(DISTINCT session_id) < 3
WHERE type = 'chat_message' AND student_id = ?
AND created_at > now() - interval '7 days'

-- Calculated at query time, no separate flag column needed
```

DB SCHEMA — ANALYTICS_DB

class_assignments			
teacher_id	UUID	from JWT	
student_id	UUID		
PRIMARY KEY		(teacher_id, student_id)	
events			
id	UUID PK		
type	VARCHAR	chat_message image_generated feedback_subm	
student_id	UUID		
school_id	UUID		
subject	VARCHAR	nullable	
session_id	UUID	nullable	
metadata	JSONB		
created_at	TIMESTAMP		
attendance			
student_id	UUID		
date	DATE		
present	BOOLEAN		
marked_by	UUID	teacher id	
PRIMARY KEY		(student_id, date)	
scores			
id	UUID PK		
student_id	UUID		
subject	VARCHAR		
score	NUMERIC	0-100	
test_date	DATE		
entered_by	UUID	teacher id · verified against class_assignments	

DEMO SEED DATA

```
// Runs on boot if events table is empty
class_assignments: teacher -> all 3 demo students
events: 30 days of chat_message events per student
attendance: 30 days · ~85% present rate
scores: Math, Science, English for each student
intervention: rahul flagged (low sessions last 7 days)
```

Tech: Node.js · Express · Prisma · All dashboard queries scope by school_id

ENV: `DATABASE_URL` · `JWT_SECRET` · `PORT=3004`

LLD v3 — Infrastructure [Docker Compose](#) · [Traefik](#) · [Ollama](#) · [ComfyUI](#)

v3: Traefik JWT plugin removed · depends_on service_healthy · ComfyUI SD model script · .env.example · ollama-init.sh at repo root

TRAEFIK/TRAEFIK.YML **CI FIXED — JWT REMOVED**

```
api:
  dashboard: true    # http://localhost:8080
  insecure: true

entryPoints:
  web:
    address: ":80"

providers:
  docker:
    exposedByDefault: false

http:
  middlewares:
    # CORS only — JWT is verified inside each service
    cors:
      headers:
        accessControlAllowOriginList:
          - "http://localhost:3000"
        accessControlAllowMethods:
          - GET
          - POST
          - OPTIONS
        accessControlAllowHeaders:
          - "Content-Type"
        accessControlAllowCredentials: true

  routers:
    auth:
      rule: "PathPrefix(`/api/auth`)"
      service: auth
      middlewares: [cors]

    ai:
      rule: "PathPrefix(`/api/ai`)"
      service: ai
      middlewares: [cors]

    rag:
      rule: "PathPrefix(`/api/rag`)"
      service: rag
      middlewares: [cors]

    analytics:
      rule: "PathPrefix(`/api/analytics`)"
      service: analytics
      middlewares: [cors]

  services: # Required — tells Traefik where to forward each route
    auth:
      loadBalancer:
        servers:
          - url: "http://auth:3001"
    ai:
      loadBalancer:
        servers:
          - url: "http://ai:3002"
    rag:
      loadBalancer:
        servers:
          - url: "http://rag:3003"
    analytics:
      loadBalancer:
        servers:
          - url: "http://analytics:3004"
```

OLLAMA-INIT.SH **MOVED TO REPO ROOT**

```
#!/bin/sh
# Located at ./ollama-init.sh in repo root (not ./services/)

# Start Ollama server in background
ollama serve &

# Wait until Ollama API responds
echo "Waiting for Ollama to start..."
until curl -sf http://localhost:11434/api/tags > /dev/null; do
  sleep 2
done
echo "Ollama ready."

# Pull models only if not already downloaded
ollama pull qwen2.5
ollama pull nomic-embed-text

echo "Models ready. Ollama running."

# Keep container alive
wait
```

✅ Traefik does routing + CORS only. Services block tells Traefik the internal Docker hostname:port for each service. Each service verifies JWT via its own middleware (see Auth LLD v3). No plugins needed.

COMFYUI-MODEL-DOWNLOAD.SH C3 NEW — RUN ONCE BEFORE DEMO

```
#!/bin/bash
# Downloads Stable Diffusion v1.5 checkpoint into ComfyUI models volume
# Run ONCE before demo day: bash comfyui-model-download.sh

echo "Downloading Stable Diffusion v1.5 checkpoint (~4GB)..."

# Create volume mountpoint if running outside Docker
docker run --rm \
  -v comfy_models:/models \
  curlimages/curl:latest \
  -L -o /models/checkpoints/v1-5-pruned-emaonly.ckpt \
  "https://huggingface.co/runwayml/stable-diffusion-v1-5/resolve/main/v1-5-pruned-emaonly.ckpt"

echo "Download complete. ComfyUI can now generate images."
# After this, docker-compose up will find the checkpoint in the named volume
```

⚠️ Run `bash comfyui-model-download.sh` once BEFORE running docker-compose up for the first time. This is separate from the main startup — do it the night before the demo. ~4GB download.

DOCKER-COMPOSE.YML — FINAL V3 C2 FIXED — SERVICE_HEALTHY

```
version: '3.9'

services:

  frontend:
    build: ./frontend
    ports: ["3000:3000"]
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:80
    depends_on:
      traefik:
        condition: service_started

  traefik:
    image: traefik:v3.0
    ports: ["80:80", "8080:8080"]
    volumes:
      - ./traefik/traefik.yml:/etc/traefik/traefik.yml
      - /var/run/docker.sock:/var/run/docker.sock

  auth:
    build: ./services/auth
    command: sh -c "npx prisma db push && node server.js" # creates tables then starts
    environment:
      DATABASE_URL: postgresql://postgres:${DB_PASSWORD}@postgres:5432/roognis?schema=auth_db
      JWT_SECRET: ${JWT_SECRET}
      DEMO_SCHOOL_ID: ${DEMO_SCHOOL_ID}
      PORT: 3001
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3001/health"]
      interval: 10s
      retries: 5
      start_period: 20s
    labels:
      - "traefik.enable=true"
    depends_on:
      postgres:
        condition: service_healthy # -- waits for pg_isready

  ai:
    build: ./services/ai
    command: sh -c "npx prisma db push && node server.js" # creates tables then starts
    environment:
      OLLAMA_URL: http://ollama:11434
      COMFYUI_URL: http://comfyui:8188
      RAG_SERVICE_URL: http://rag:3003
```

```

ANALYTICS_URL: http://analytics:3004
DATABASE_URL: postgresql://postgres:${DB_PASSWORD}@postgres:5432/roognis?schema=ai_db
FILE_STORAGE_PATH: /data
JWT_SECRET: ${JWT_SECRET}
PORT: 3002
volumes:
  - file_storage:/data
  - ./seed-data:/seed-data:ro # videos served from /seed-data/videos/
healthcheck:
  test: ["CMD","curl","-f","http://localhost:3002/health"]
  interval: 10s  retries: 5  start_period: 20s
depends_on:
  postgres:
    condition: service_healthy
  ollama:
    condition: service_healthy # -- waits until qwen2.5 is fully pulled
  rag:
    condition: service_healthy

rag:
build: ./services/rag
# RAG is Python - uses SQLAlchemy Base.metadata.create_all() on startup (see rag/main.py)
environment:
  OLLAMA_URL: http://ollama:11434
  CHROMA_URL: http://chromadb:8000
  DATABASE_URL: postgresql://postgres:${DB_PASSWORD}@postgres:5432/roognis?schema=rag_db
  FILE_STORAGE_PATH: /data
  DEMO_SCHOOL_ID: ${DEMO_SCHOOL_ID}
  JWT_SECRET: ${JWT_SECRET}
  PORT: 3003
volumes:
  - file_storage:/data
  - ./seed-data:/seed-data:ro # NCERT PDFs mounted read-only
healthcheck:
  test: ["CMD","curl","-f","http://localhost:3003/health"]
  interval: 10s  retries: 5  start_period: 20s
depends_on:
  postgres:
    condition: service_healthy
  chromadb:
    condition: service_started
  ollama:
    condition: service_healthy

analytics:
build: ./services/analytics
command: sh -c "npm prisma db push && node server.js" # creates tables then starts
environment:
  DATABASE_URL: postgresql://postgres:${DB_PASSWORD}@postgres:5432/roognis?schema=analytics_db
  JWT_SECRET: ${JWT_SECRET}
  PORT: 3004
healthcheck:
  test: ["CMD","curl","-f","http://localhost:3004/health"]
  interval: 10s  retries: 5  start_period: 20s
depends_on:
  postgres:
    condition: service_healthy

postgres:
image: postgres:16
environment:
  POSTGRES_DB: roognis
  POSTGRES_PASSWORD: ${DB_PASSWORD}
volumes:
  - postgres_data:/var/lib/postgresql/data
healthcheck:
  test: ["CMD-SHELL","pg_isready -U postgres"]
  interval: 5s  retries: 10

chromadb:
image: chromadb/chroma:latest
volumes:
  - chroma_data:/chroma/chroma

ollama:
image: ollama/ollama:latest
volumes:
  - ollama_models:/root/.ollama

```

```
- ./ollama-init.sh:/init.sh # at repo root (not ./services/)
entrypoint: ["sh", "/init.sh"]
healthcheck:
  test: ["CMD-SHELL", "ollama list 2>/dev/null | grep -q qwen2.5"]
  # : passes ONLY after qwen2.5 is fully pulled - not just when API starts
  interval: 30s  retries: 20  start_period: 120s
```

comfyui:

```
image: ghcr.io/ai-dock/comfyui:latest
ports: ["8188:8188"]
volumes:
  - comfy_models:/opt/ComfyUI/models # SD checkpoint mounted here after model download
  - file_storage:/data
```

volumes:

```
postgres_data:
chroma_data:
ollama_models:
comfy_models:
file_storage:
```

.ENV.EXAMPLE W3 NEW — COMMIT TO REPO

```
# Copy to .env before running: cp .env.example .env
# Change JWT_SECRET and DB_PASSWORD before any real deployment
```

```
JWT_SECRET=roognis-super-secret-change-me
DB_PASSWORD=postgres123
DEMO_SCHOOL_ID=550e8400-e29b-41d4-a716-446655440000
```

✅ Commit this file to the repo. Do NOT commit .env (add to .gitignore).

STARTUP SEQUENCE — DEMO DAY

```
# 1. Clone repo + create .env
cp .env.example .env

# 2. ONE-TIME: Download ComfyUI SD model (~4GB)
#   Do this the night before demo day
bash comfyui-model-download.sh

# 3. First time setup (night before demo)
docker-compose up --build
# - Downloads Ollama models (~8GB), seeds DB
# - Embeds NCERT chapters into ChromaDB
# - Takes 30-60 mins total on first run

# 4. Demo day (30s before investors arrive)
docker-compose up
# - All services ready in ~30s (models already cached)
# - open http://localhost:3000
# - login: teacher@demo.com / demo1234
# - login: arjun@demo.com / demo1234 (student)
# - login: parent1@demo.com / demo1234 (parent)
```